Python implementation

Here is a simple Python implementation of the above ABSA approach for the review text: "I bought a spectacular camera but it has a terrible battery."

```
import os
os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"
# import your ABSA or Transformers libraries
from transformers import AutoModel
import spacy
from transformers import pipeline
# Load SpaCy for Aspect/Opinion Extraction (Step 1 & 3)
nlp = spacy.load("en_core_web_sm")
#Step 2
# Load a pre-trained BERT pipeline for Sentiment
# Load a model specifically fine-tuned for ABSA or product
reviews
# device=0 means the first GPU, device=-1 means CPU
import torch
# This checks if a GPU is available and set the device
accordingly
device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
print(f"Using device: {device}")
d = 0 if torch.cuda.is_available() else -1
sentiment_analyzer = pipeline(
    "sentiment-analysis",
    model="LiYuan/amazon-review-sentiment-analysis",
    device=d
)
#sentiment_analyzer = pipeline("sentiment-analysis",
model="LiYuan/amazon-review-sentiment-analysis")
def run_absa_pipeline(text):
    doc = nlp(text)
    triplets = []
    # STEP 1:
    # Aspect Extraction (Finding Nouns) ---
    # We look for nouns as potential aspects
    aspects = [token for token in doc if token.pos_ in
["NOUN", "PROPN"]]
    for aspect in aspects:
        # STEP 3 (Part A):
       # Finding Opinion Terms ---
        # We look for adjectives (opinions) connected to the
aspect in the grammar tree
        opinions = [child.text for child in aspect.children
if child.pos_ == "ADJ"]
        if not opinions:
                  continue # Skip if no descriptive word is
found
        for opinion in opinions:
                 # --- STEP 2: Sentiment Classification ---
          # We construct an "Auxiliary Sentence" for BERT:
[Aspect] is [Opinion]
             # This helps BERT focus its context.
             aux_sentence = f"{aspect.text} is {opinion}"
             result = sentiment_analyzer(aux_sentence)[0]
             sentiment = result['label']
             # --- STEP 3 (Part B): Final Triplet
Extraction ---
             triplets.append({
                "aspect": aspect.text,
                "opinion": opinion,
                "sentiment": sentiment
             })
    return triplets
# Test the pipeline
review = "I bought a spectacular camera but it has a terrible
battery."
results = run_absa_pipeline(review)
print(f"Review: {review}\n")
print(f"{'Aspect':<15} | {'Opinion':<15} | {'Sentiment'}")
print("-" * 45)
for t in results:
    print(f"{t['aspect']:<15} | {t['opinion']:<15} |
{t['sentiment']}")
```

The output is:

```
Using device: cuda
Device set to use cuda:0
Review: I bought a spectacular camera but it has a terrible
battery.
Aspect          | Opinion         | Sentiment
-------------------------------------------------
camera          | spectacular     | 5 stars
battery         | terrible        | 1 star
```

Note: This ABSA model is review text-dependent; make a simple sentence to have prompt sentiment analysis.

Limitations

Although BERT has revolutionised NLP and is widely used in modern ABSA, it isn't a faultless model.

Size limitation: Standard BERT cannot process sequences longer than 512 tokens; it simply truncates anything beyond this limit.

Memory limitation: As BERT uses quadratic self-attention, for every doubling of the length of the input the computational cost increases fourfold.

■■ *Continue to page... 79*